



Clase 40 - Practica

Práctica guiada — De los orígenes al informe: medallion + star schema en Microsoft Fabric

0. Preparación

0.1 Requisitos previos

Requisito	Detalle
Capacidad Fabric	Trial, F2 o superior (para el paso de <i>Semantic model refresh</i> en pipeline se requiere Premium, PPU o Embedded)
Permisos	Rol Admin o Member en el workspace
Power BI Desktop	Opcional; toda la práctica se puede hacer en el navegador

0.2 Arquitectura que vamos a construir

Aplicamos el **Patrón 2** de Microsoft: Bronze y Silver como lakehouses, **Gold como Warehouse**.

Capa	Ítem de Fabric	Nombre	Contenido
Bronze	Lakehouse	LH_Bronze	CSV crudos en Files/
Silver	Lakehouse	LH_Silver	Tablas Delta limpias
Gold	Warehouse	WH_Gold	Star schema en T-SQL
Orquestación	Data pipeline	PL_Medallion	Encadena todo
Capa semántica	Semantic model	SM_Ventas	Direct Lake sobre WH_Gold

Capa	Ítem de Fabric	Nombre	Contenido
Consumo	Report	RPT_Ventas	Informe de Power BI

🔗 Usamos **un solo workspace** para simplificar la práctica. En producción, Microsoft recomienda **un workspace por capa** para mejorar el control y el gobierno.

0.3 Crear el workspace y los ítems

1. En Fabric, **Workspaces** → **New workspace**. Nombre: `WS-DP600-Lab`.
2. Dentro del workspace, **New item** → **Lakehouse** → nombre `LH_Bronze`. Deja **Lakehouse schemas** desactivado.
3. Repite → `LH_Silver`.
4. **New item** → **Warehouse** → nombre `WH_Gold`.

ℹ Desde el 5 de septiembre de 2025, Fabric **ya no crea automáticamente** un *default semantic model* al crear un warehouse o lakehouse. Lo crearemos a mano en la parte 5.

Punto de control 0: en el workspace debes ver tres ítems (`LH_Bronze`, `LH_Silver`, `WH_Gold`) más los dos *SQL analytics endpoint* asociados a los lakehouses.

1. Capa Bronze — generar e ingerir los datos crudos

En un proyecto real, aquí correría un pipeline con *Copy activity* o un **shortcut**. Para que la práctica sea reproducible sin depender de orígenes externos, **generamos los datos con PySpark** y los escribimos como CSV en `Files/`, simulando lo que llegaría de un sistema de origen.

Los datos incluyen **suciedad deliberada** — duplicados, nulos, formatos de fecha mezclados, transacciones de prueba — que limpiaremos en Silver.

1.1 Crear el notebook de ingesta

1. **New item** → **Notebook** → nombre `NB_01_Bronze_Ingesta`.
2. En el panel **Explorer** izquierdo, **Add data items** → **From OneLake Catalog** → selecciona `LH_Bronze`. Debe quedar como lakehouse por defecto.
3. Pega y ejecuta la celda siguiente.

```
# NB_01_Bronze_Ingesta
# Genera los ficheros crudos del sistema de origen y los deja en Files/raw/

from pyspark.sql import Row
import random, datetime

random.seed(42)
WORKSPACE = "WS-DP600-Lab" # ya sé que este es tu nombre exacto de worksp
```

```

ace
BASE = (f"abfss://{WORKSPACE}@onelake.dfs.fabric.microsoft.com/"
        f"LH_Bronze.Lakehouse/Files/raw")
# ----- Origen 1: catálogo de productos (CRM) -----
productos = [
    ("P001", "Portátil Zeta 14", "Portátiles", "Informática", "Contoso", 8
99.00),
    ("P002", "Portátil Zeta 16", "Portátiles", "Informática", "Contoso", 1
199.00),
    ("P003", "Ratón Óptico M2", "Periféricos", "Informática", "Contoso", 2
4.90),
    ("P004", "Teclado Mecánico", "Periféricos", "Informática", "Contoso", 8
9.00),
    ("P005", "Monitor 27 QHD", "Monitores", "Informática", "Contoso", 3
29.00),
    ("P006", "Silla Ergo Pro", "Mobiliario", "Oficina", "Fabrikam",
449.00),
    ("P007", "Mesa Elevable", "Mobiliario", "Oficina", "Fabrikam",
599.00),
    ("P008", "Lámpara LED Flex", "Iluminación", "Oficina", "Fabrikam",
39.90),
    # Duplicado deliberado de P003 (mismo producto cargado dos veces)
    ("P003", "Ratón Óptico M2", "Periféricos", "Informática", "Contoso", 2
4.90),
    # Fila con nulos deliberados
    ("P009", "Alfombrilla XL", None, "Oficina", None, 1
2.50),
]
df_prod = spark.createDataFrame(productos,
    "ProductCode string, ProductName string, Subcategory string, Category s
tring, Brand string, ListPrice double")

# ----- Origen 2: clientes (ERP) -----
clientes = [
    ("C001", "Ana", "García", "Madrid", "España", "Retail", "a
na.garcia@example.com"),
    ("C002", "Bruno", "Lima", "Barcelona", "España", "Corporate", "b
runo.lima@example.com"),
    ("C003", "Carla", "Núñez", "Valencia", "España", "Retail", "c
arla.nunez@example.com"),
    ("C004", "Diego", "Ferrer", "Sevilla", "España", "Corporate", No
ne),
    ("C005", "Elena", "Ruiz", "Bilbao", "España", "Retail", "e

```

```

lena.ruiz@example.com"),
    ("C006", "Fabio", "Costa", "Lisboa", "Portugal", "Corporate", "fabio.costa@example.com"),
]
df_cli = spark.createDataFrame(clientes,
    "CustomerCode string, FirstName string, LastName string, City string, Country string, Segment string, Email string")

# ----- Origen 3: tiendas -----
tiendas = [
    ("S01", "Contoso Gran Vía", "Madrid", "Centro", "Física"),
    ("S02", "Contoso Diagonal", "Barcelona", "Este", "Física"),
    ("S03", "Contoso Online", "Madrid", "Online", "Online"),
]
df_tie = spark.createDataFrame(tiendas,
    "StoreCode string, StoreName string, City string, Region string, Channel string")

# ----- Origen 4: líneas de pedido (POS) -----
codigos_prod = [p[0] for p in productos[:8]]
codigos_cli = [c[0] for c in clientes]
codigos_tie = [t[0] for t in tiendas]
precios = {p[0]: p[5] for p in productos[:8]}

filas = []
inicio = datetime.date(2025, 1, 1)
for i in range(1, 1201):
    order_date = inicio + datetime.timedelta(days=random.randint(0, 545))
    ship_date = order_date + datetime.timedelta(days=random.randint(1, 7))
    pc = random.choice(codigos_prod)
    qty = random.randint(1, 5)
    precio = precios[pc]
    desc = round(precio * qty * random.choice([0, 0, 0, 0.05, 0.10, 0.15]), 2)
    filas.append((
        f"ORD-{i:05d}", # OrderNumber (degenerate dimension)
        f"{i:05d}-1", # OrderLine
        order_date.strftime("%Y-%m-%d"), # formato ISO
        ship_date.strftime("%d/%m/%Y"), # formato distinto a propósito
        pc,
        random.choice(codigos_cli),
        random.choice(codigos_tie),
    ))

```



```

        qty,
        precio,
        desc,
    ))

# Suciedad deliberada: 15 líneas duplicadas y 10 pedidos de prueba
filas += random.sample(filas, 15)
for i in range(10):
    filas.append((f"TEST-{i:03d}", "1", "2025-06-15", "16/06/2025",
                  "P001", "C001", "S01", 1, 899.00, 0.0))

df_ven = spark.createDataFrame(filas,
    "OrderNumber string, OrderLine string, OrderDate string, ShipDate string, "
    "ProductCode string, CustomerCode string, StoreCode string, "
    "Quantity int, UnitPrice double, DiscountAmount double")

# ----- Escritura en Bronze (formato original: CSV) -----
for nombre, df in [("productos", df_prod), ("clientes", df_cli),
                  ("tiendas", df_tie), ("ventas", df_ven)]:
    (df.coalesce(1).write.mode("overwrite")
     .option("header", True).csv(f"{BASE}/{nombre}"))
    print(f"✓{nombre}:{df.count()} filas escritas en{BASE}/{nombre}")

```

Punto de control 1: en `LH_Bronze → Files → raw` deben aparecer cuatro carpetas. Ventas debe tener **1.225 filas** (1.200 + 15 duplicados + 10 de prueba).

💡 **Principio Bronze:** *store everything exactly as it arrives, no changes are allowed.* No hemos corregido nada: los duplicados, los nulos y los dos formatos de fecha siguen ahí. Esa es la definición de la capa.

🔑 **En un proyecto real,** si el origen ya estuviera en OneLake, ADLS Gen2, Amazon S3 o Google Cloud Storage, la recomendación de Microsoft es **crear un shortcut en Bronze en vez de copiar los datos.**

2. Capa Silver — limpiar y conformar

Objetivo: corregir errores, estandarizar formatos, eliminar duplicados y escribir **tablas Delta**.

2.1 Crear el notebook de transformación

1. **New item → Notebook** → nombre `NB_02_Silver_Limpieza`.
2. Añade `LH_Silver` como lakehouse por defecto en el Explorer.
3. Ejecuta las celdas siguientes.

```

# NB_02_Silver_Limpieza - Celda 1: leer desde Bronze
# Ruta ABFS al lakehouse Bronze (sustituye <WORKSPACE> por el nombre de tu
workspace)
BRONZE = "abfss://WS-DP600-Lab@onelake.dfs.fabric.microsoft.com/LH_Bronze.L
akehouse/Files/raw"

def leer(nombre):
    return (spark.read.option("header", True).option("inferSchema", True)
            .csv(f"{BRONZE}/{nombre}"))

b_prod = leer("productos")
b_cli = leer("clientes")
b_tie = leer("tiendas")
b_ven = leer("ventas")

print(b_ven.count(), "líneas en bronze")

```

⚠ Si prefieres no escribir la ruta ABFS, añade también `LH_Bronze` al Explorer del notebook y usa el path relativo del lakehouse no predeterminado. La ruta ABFS es más explícita y menos frágil, por eso la usamos aquí.

```

# Celda 2: limpiar productos y clientes
from pyspark.sql import functions as F

s_prod = (b_prod
          .dropDuplicates(["ProductCode"]) # duplicado de
P003
          .withColumn("Subcategory", F.coalesce("Subcategory", F.lit("Sin subcate
goría"))))
          .withColumn("Brand", F.coalesce("Brand", F.lit("Sin marc
a"))))
          .withColumn("ListPrice", F.col("ListPrice").cast("decimal(10,2)"))
          )

s_cli = (b_cli
         .dropDuplicates(["CustomerCode"])
         .withColumn("Email", F.coalesce("Email", F.lit("desconocido@example.co
m"))))
         .withColumn("FullName", F.concat_ws(" ", "FirstName", "LastName"))
         )

s_tie = b_tie.dropDuplicates(["StoreCode"])

```

```
display(s_prod)
```

```
# Celda 3: limpiar ventas – el trabajo de verdad
s_ven = (b_ven
# 1. eliminar duplicados exactos de línea de pedido
.dropDuplicates(["OrderNumber", "OrderLine"])
# 2. eliminar pedidos de prueba
.filter(~F.col("OrderNumber").startswith("TEST-"))
# 3. estandarizar los DOS formatos de fecha a tipo date
.withColumn("OrderDate", F.to_date("OrderDate", "yyyy-MM-dd"))
.withColumn("ShipDate", F.to_date("ShipDate", "dd/MM/yyyy"))
# 4. tipar los importes con decimal (nunca float para dinero)
.withColumn("Quantity", F.col("Quantity").cast("int"))
.withColumn("UnitPrice", F.col("UnitPrice").cast("decimal(10,2)"))
.withColumn("DiscountAmount", F.col("DiscountAmount").cast("decimal(10,2)"))
# 5. medida derivada: importe bruto y neto
.withColumn("GrossAmount", (F.col("Quantity") * F.col("UnitPrice")).cast("decimal(12,2)"))
.withColumn("NetAmount", (F.col("Quantity") * F.col("UnitPrice") - F.col("DiscountAmount")).cast("decimal(12,2)"))
# 6. descartar filas sin fecha o sin producto (integridad mínima)
.filter(F.col("OrderDate").isNotNull() & F.col("ProductCode").isNotNull())
)

print("Silver ventas:", s_ven.count(), "filas (esperado: 1200)")
display(s_ven.limit(10))
```

```
# Celda 4: escribir las tablas Delta de Silver
for nombre, df in [("dim_producto_src", s_prod), ("dim_cliente_src", s_cli),
                  ("dim_tienda_src", s_tie), ("ventas", s_ven)]:
    df.write.mode("overwrite").format("delta").saveAsTable(nombre)
print(f"✓ Tabla Delta creada:{nombre}")
```

Punto de control 2: `LH_Silver → Tables` debe mostrar cuatro tablas Delta. `ventas` debe tener **1.200 filas** (se eliminaron 15 duplicados y 10 de prueba).

 **Por qué decimal y no float para importes:** `float` es un tipo aproximado; sumar millones de importes acumula error de redondeo. Es un error de diseño que aparece en auditorías reales.

Además, `decimal` es un tipo soportado tanto en Delta como en Fabric Warehouse.

3. Capa Gold — construir el star schema en el Warehouse

Aquí aplicamos **todo lo visto clases anteriores**: surrogate keys, special dimension members, SCD tipo 1 y tipo 2, degenerate dimension y role-playing dimension.

Abre `WH_Gold` → **New SQL query**

3.1 Crear el esquema y las tablas

```
-- =====
-- 3.1 DDL del star schema
-- =====

CREATE SCHEMA gold;
GO

-- Dimensión de fecha (surrogate key con significado: YYYYMMDD)
CREATE TABLE gold.Dim_Date (
    DateKey          int          NOT NULL,
    FullDate         date         NOT NULL,
    Year            smallint      NOT NULL,
    Quarter         smallint      NOT NULL,
    QuarterName     varchar(2)   NOT NULL,
    Month           smallint      NOT NULL,
    MonthName       varchar(20)  NOT NULL,
    YearMonth       varchar(7)   NOT NULL,
    Day            smallint      NOT NULL,
    DayOfWeek       smallint      NOT NULL,
    DayName         varchar(20)  NOT NULL,
    IsWeekend       bit          NOT NULL
);

-- Dimensión de producto → SCD tipo 1 (sobrescribe)
CREATE TABLE gold.Dim_Product (
    Product_SK      bigint        NOT NULL,    -- surrogate key
    ProductCode     varchar(10)   NOT NULL,    -- natural key
    ProductName     varchar(100)  NOT NULL,
    Subcategory     varchar(50)   NOT NULL,
    Category        varchar(50)   NOT NULL,
    Brand           varchar(50)   NOT NULL,
    ListPrice       decimal(10,2) NULL,
    RecUpdatedDate  datetime2(6)  NOT NULL    -- audit attribute
);
```

```

-- Dimensión de cliente → SCD tipo 2 (versiona el histórico)
CREATE TABLE gold.Dim_Customer (
    Customer_SK      bigint      NOT NULL,
    CustomerCode     varchar(10)  NOT NULL,
    FullName         varchar(100) NOT NULL,
    City             varchar(50)  NOT NULL,    -- atributo con seguimiento his
tórico
    Country          varchar(50)  NOT NULL,
    Segment          varchar(30)  NOT NULL,
    Email            varchar(100) NOT NULL,    -- atributo SCD tipo 1
    RecStartDate     date          NOT NULL,    -- historical tracking attribut
es
    RecEndDate       date          NOT NULL,
    RecIsCurrent     bit           NOT NULL,
    RecUpdatedDate   datetime2(6) NOT NULL
);

-- Dimensión de tienda → SCD tipo 1
CREATE TABLE gold.Dim_Store (
    Store_SK        bigint      NOT NULL,
    StoreCode       varchar(10)  NOT NULL,
    StoreName       varchar(100) NOT NULL,
    City            varchar(50)  NOT NULL,
    Region          varchar(50)  NOT NULL,
    Channel         varchar(20)  NOT NULL,
    RecUpdatedDate  datetime2(6) NOT NULL
);

-- Tabla de hechos → grain: UNA FILA POR LÍNEA DE PEDIDO
CREATE TABLE gold.Fact_Sales (
-- dimension keys
    OrderDateKey    int          NOT NULL,    -- role-playing dimension
    ShipDateKey     int          NOT NULL,    -- role-playing dimension
    Product_SK      bigint      NOT NULL,
    Customer_SK     bigint      NOT NULL,
    Store_SK        bigint      NOT NULL,
-- attributes (degenerate dimension)
    OrderNumber     varchar(20)  NOT NULL,
    OrderLine       varchar(10)  NOT NULL,
-- measures
    Quantity        int          NOT NULL,
    UnitPrice       decimal(10,2) NOT NULL,
    GrossAmount     decimal(12,2) NOT NULL,

```

```

DiscountAmount decimal(10,2) NOT NULL,
NetAmount      decimal(12,2) NOT NULL,
-- audit
RecLoadedDate  datetime2(6)  NOT NULL
);
GO

```

Verificación rápida al terminar:

```

SELECT s.name AS Esquema, t.name AS Tabla
FROM sys.tables t
JOIN sys.schemas s ON s.schema_id = t.schema_id
WHERE s.name = 'gold'
ORDER BY t.name;

```

Deberías ver las cinco tablas: `Dim_Date`, `Dim_Product`, `Dim_Customer`, `Dim_Store`, `Fact_Sales`.

⚙️ **Sobre las surrogate keys.** Usamos `bigint` poblado con `ROW_NUMBER()` en lugar de `IDENTITY` porque `IDENTITY` en Fabric Data Warehouse está **en preview**, solo admite `bigint`, no permite `IDENTITY_INSERT` ni configurar `SEED` / `INCREMENT`, y **no garantiza el orden ni la ausencia de huecos** (asigna rangos distintos por nodo de cómputo). Para una práctica reproducible, `ROW_NUMBER()` es más predecible. En producción cualquiera de las dos opciones es válida.

3.2 Poblar la dimensión de fecha

Fabric Data Warehouse **no soporta CTE recursivas**, así que generamos el calendario con un *cross join* de listas de valores.

```

-- =====
-- 3.2 Dim_Date (2024-01-01 a 2027-12-31)
-- =====
WITH d0 AS (SELECT 0 AS n UNION ALL SELECT 1 UNION ALL SELECT 2 UNION ALL S
ELECT 3
            UNION ALL SELECT 4 UNION ALL SELECT 5 UNION ALL SELECT 6 UNION
ALL SELECT 7
            UNION ALL SELECT 8 UNION ALL SELECT 9),
nums AS (SELECT (a.n + b.n*10 + c.n*100 + e.n*1000) AS i
          FROM d0 a CROSS JOIN d0 b CROSS JOIN d0 c CROSS JOIN d0 e),
cal AS (SELECT DATEADD(day, i, CAST('2024-01-01' AS date)) AS FullDate
        FROM nums WHERE i <= 1460)
INSERT INTO gold.Dim_Date
SELECT
    CAST(FORMAT(FullDate, 'yyyyMMdd') AS int) AS Date
Key,
FullDate,

```

```

        YEAR(FullDate) AS Yea
r,
        DATEPART(quarter, FullDate) AS Quar
ter,
        'Q' + CAST(DATEPART(quarter, FullDate) AS varchar(1)) AS Quar
terName,
        MONTH(FullDate) AS Mont
h,
        DATENAME(month, FullDate) AS Mont
hName,
        FORMAT(FullDate, 'yyyy-MM') AS Year
Month,
        DAY(FullDate) AS Day,
        DATEPART(weekday, FullDate) AS Day0
fWeek,
        DATENAME(weekday, FullDate) AS DayN
ame,
        CASE WHEN DATEPART(weekday, FullDate) IN (1,7) THEN 1 ELSE 0 END AS IsW
eekend
FROM cal;

-- Special dimension member: fecha desconocida
INSERT INTO gold.Dim_Date VALUES
(-1, '1900-01-01', 1900, 1, 'Q1', 1, 'Desconocido', '1900-01', 1, 1, 'Desc
onocido', 0);

SELECT COUNT(*) AS FilasDimDate FROM gold.Dim_Date; -- esperado: 1462

```

17 **Por qué YYYYMMDD**. Es la excepción aceptada a la regla de "no dar significado a las surrogate keys": la clave es legible, ocupa un `int` y, sobre todo, **se puede calcular** durante la carga del fact sin necesidad de lookup.

3.3 Cargar las dimensiones con special members

El Warehouse lee las tablas Delta de `LH_Silver` mediante **consulta cross-database con nomenclatura de tres partes**. Requisito: ambos ítems en el **mismo workspace y misma región**.

1. En el Explorer de `WH_Gold`, pulsa + **Warehouses** y añade el **SQL analytics endpoint** de `LH_Silver`.
2. Ejecuta:

```

-- =====
-- 3.3 Dim_Product - SCD tipo 1 (carga inicial)
-- =====
-- Special dimension member para lookups fallidos

```



```
INSERT INTO gold.Dim_Product
VALUES (-1, 'N/A', 'Desconocido', 'Desconocido', 'Desconocido', 'Desconocido', NULL, SYSDATETIME());
```

```
INSERT INTO gold.Dim_Product
SELECT
    ROW_NUMBER() OVER (ORDER BY s.ProductCode) AS Product_SK,
    s.ProductCode, s.ProductName, s.Subcategory, s.Category, s.Brand,
    CAST(s.ListPrice AS decimal(10,2)),
    SYSDATETIME()
FROM LH_Silver.dbo.dim_producto_src AS s;
```

```
-- =====
--      Dim_Store - SCD tipo 1
-- =====
```

```
INSERT INTO gold.Dim_Store
VALUES (-1, 'N/A', 'Desconocido', 'Desconocido', 'Desconocido', 'Desconocido', SYSDATETIME());
```

```
INSERT INTO gold.Dim_Store
SELECT
    ROW_NUMBER() OVER (ORDER BY s.StoreCode) AS Store_SK,
    s.StoreCode, s.StoreName, s.City, s.Region, s.Channel, SYSDATETIME()
FROM LH_Silver.dbo.dim_tienda_src AS s;
```

```
-- =====
--      Dim_Customer - SCD tipo 2 (carga inicial: todo vigente)
-- =====
```

```
INSERT INTO gold.Dim_Customer
VALUES (-1, 'N/A', 'Desconocido', 'Desconocido', 'Desconocido', 'Desconocido',
        'n/a', '1900-01-01', '9999-12-31', 1, SYSDATETIME());
```

```
INSERT INTO gold.Dim_Customer
SELECT
    ROW_NUMBER() OVER (ORDER BY s.CustomerCode) AS Customer_SK,
    s.CustomerCode, s.FullName, s.City, s.Country, s.Segment, s.Email,
    CAST('2024-01-01' AS date) AS RecStartDate,
    CAST('9999-12-31' AS date) AS RecEndDate,
    1 AS RecIsCurrent,
    SYSDATETIME()
FROM LH_Silver.dbo.dim_cliente_src AS s;
```

```
-- Verificación
SELECT 'Product' AS Dim, COUNT(*) AS Filas FROM gold.Dim_Product
UNION ALL SELECT 'Store', COUNT(*) FROM gold.Dim_Store
UNION ALL SELECT 'Customer', COUNT(*) FROM gold.Dim_Customer;
```

Punto de control 3: Product = 10 (9 + Unknown), Store = 4, Customer = 7.

 **Special dimension members.** La convención de Microsoft usa `0` = Missing, `-1` = Unknown, `-2` = N/A, `-3` = Error. Aquí usamos `-1` para todo por simplicidad. Su función es permitir que **todas las dimension keys del fact sean NOT NULL** sin perder filas de hechos cuando un lookup falla.

3.4 Cargar la tabla de hechos

Las dimensiones ya están cargadas — ahora y solo ahora se carga el fact.

```
-- =====
-- 3.4 Fact_Sales - grain: una fila por línea de pedido
-- =====

INSERT INTO gold.Fact_Sales
SELECT
-- La clave de fecha se CALCULA, no se busca (ventaja del formato YYYYMMDD)
ISNULL(d0rd.DateKey, -1) AS OrderDateKey,
ISNULL(dShp.DateKey, -1) AS ShipDateKey,
-- El resto se resuelve por lookup contra la versión vigente
ISNULL(p.Product_SK, -1) AS Product_SK,
ISNULL(c.Customer_SK, -1) AS Customer_SK,
ISNULL(t.Store_SK, -1) AS Store_SK,
v.OrderNumber,
v.OrderLine,
CAST(v.Quantity AS int),
CAST(v.UnitPrice AS decimal(10,2)),
CAST(v.GrossAmount AS decimal(12,2)),
CAST(v.DiscountAmount AS decimal(10,2)),
CAST(v.NetAmount AS decimal(12,2)),
SYSDATETIME()
FROM LH_Silver.dbo.ventas AS v
LEFT JOIN gold.Dim_Date AS d0rd ON d0rd.FullDate = v.OrderDate
LEFT JOIN gold.Dim_Date AS dShp ON dShp.FullDate = v.ShipDate
LEFT JOIN gold.Dim_Product AS p ON p.ProductCode = v.ProductCode
LEFT JOIN gold.Dim_Store AS t ON t.StoreCode = v.StoreCode
LEFT JOIN gold.Dim_Customer AS c ON c.CustomerCode = v.CustomerCode
AND c.RecIsCurrent = 1; -- ★ versión
VIGENTE
```

```
-- Auditoría de integridad: ¿cuántos lookups han fallado?
SELECT
    SUM(CASE WHEN Product_SK = -1 THEN 1 ELSE 0 END) AS ProductoDesconocido,
    SUM(CASE WHEN Customer_SK = -1 THEN 1 ELSE 0 END) AS ClienteDesconocido,
    SUM(CASE WHEN Store_SK = -1 THEN 1 ELSE 0 END) AS TiendaDesconocida,
    SUM(CASE WHEN OrderDateKey = -1 THEN 1 ELSE 0 END) AS FechaDesconocida,
    COUNT(*) AS TotalFilas
FROM gold.Fact_Sales;
```

Punto de control 4: 1.200 filas y **cero** lookups fallidos.

⚠ El punto crítico de toda la práctica. Fabric Warehouse **admite** foreign keys pero **no las impone**. Si el lookup falla, no salta ningún error: acabas con datos silenciosamente incorrectos. Por eso esta consulta de auditoría no es opcional — es parte del proceso de carga. En producción se convierte en un paso del pipeline que falla si el umbral se supera.

3.5 Declarar las constraints (no impuestas)

Aunque no se validen, conviene crearlas: permiten que **Power BI Desktop detecte y cree las relaciones automáticamente**.

```
-- =====
-- 3.5 Constraints NOT ENFORCED
-- =====

ALTER TABLE gold.Dim_Date      ADD CONSTRAINT PK_DimDate      PRIMARY KEY NON
CLUSTERED (DateKey)           NOT ENFORCED;
ALTER TABLE gold.Dim_Product  ADD CONSTRAINT PK_DimProduct  PRIMARY KEY NON
CLUSTERED (Product_SK)        NOT ENFORCED;
ALTER TABLE gold.Dim_Customer ADD CONSTRAINT PK_DimCustomer PRIMARY KEY NON
CLUSTERED (Customer_SK)       NOT ENFORCED;
ALTER TABLE gold.Dim_Store    ADD CONSTRAINT PK_DimStore     PRIMARY KEY NON
CLUSTERED (Store_SK)          NOT ENFORCED;

ALTER TABLE gold.Fact_Sales ADD CONSTRAINT FK_Fact_OrderDate
FOREIGN KEY (OrderDateKey) REFERENCES gold.Dim_Date(DateKey) NOT ENFORCED;
ALTER TABLE gold.Fact_Sales ADD CONSTRAINT FK_Fact_Product
FOREIGN KEY (Product_SK) REFERENCES gold.Dim_Product(Product_SK) NOT ENFORCED;
ALTER TABLE gold.Fact_Sales ADD CONSTRAINT FK_Fact_Customer
FOREIGN KEY (Customer_SK) REFERENCES gold.Dim_Customer(Customer_SK) NOT ENFORCED;
ALTER TABLE gold.Fact_Sales ADD CONSTRAINT FK_Fact_Store
```

```
FOREIGN KEY (Store_SK) REFERENCES gold.Dim_Store(Store_SK) NOT ENFORCE
D;
GO
```

i Solo se crea **una** FK hacia `Dim_Date` (la de `OrderDateKey`). La segunda relación, la de `ShipDateKey`, la definiremos en el semantic model como **relación inactiva** — es la esencia de la *role-playing dimension*.

3.6 Vista para la degenerate dimension

`OrderNumber` vive en el fact porque está **al mismo grano que los hechos**. Si el negocio necesita consultarlo como dimensión, se expone mediante una vista:

```
CREATE VIEW gold.Dim_Order AS
SELECT DISTINCT OrderNumber FROM gold.Fact_Sales;
GO
```

4. Procedimientos de carga incremental y SCD tipo 2

Hasta aquí hemos hecho una carga inicial. Ahora convertimos la lógica en **stored procedures reutilizables**, que es lo que orquestará el pipeline.

4.1 SCD tipo 1 con MERGE

`MERGE` es **generalmente disponible** en Fabric Data Warehouse desde enero de 2026.

```
-- =====
-- 4.1 SP: cargar Dim_Product (SCD tipo 1)
-- =====

CREATE PROCEDURE gold.sp_Load_Dim_Product AS
BEGIN
    MERGE gold.Dim_Product AS tgt
    USING (
        SELECT ProductCode, ProductName, Subcategory, Category, Brand,
               CAST(ListPrice AS decimal(10,2)) AS ListPrice
        FROM LH_Silver.dbo.dim_producto_src
    ) AS src
    ON tgt.ProductCode = src.ProductCode
    WHEN MATCHED AND (tgt.ProductName <> src.ProductName
                     OR tgt.Subcategory <> src.Subcategory
                     OR tgt.Brand <> src.Brand
                     OR ISNULL(tgt.ListPrice,-1) <> ISNULL(src.ListPrice,-1))
    THEN UPDATE SET
        tgt.ProductName = src.ProductName,
        tgt.Subcategory = src.Subcategory,
```

```

        tgt.Category      = src.Category,
        tgt.Brand         = src.Brand,
        tgt.ListPrice     = src.ListPrice,
        tgt.RecUpdatedDate = SYSDATETIME()
    WHEN NOT MATCHED BY TARGET
    THEN INSERT (Product_SK, ProductCode, ProductName, Subcategory, Category, Brand, ListPrice, RecUpdatedDate)
    VALUES ((SELECT MAX(Product_SK) FROM gold.Dim_Product) + 1,
            src.ProductCode, src.ProductName, src.Subcategory,
            src.Category, src.Brand, src.ListPrice, SYSDATETIME());
END;
GO

```

⚠ Ese `MAX(...) + 1` funciona para un lote pequeño como el de la práctica. Con volúmenes reales, genera la clave con `ROW_NUMBER() OVER (...) + (SELECT ISNULL(MAX(Product_SK),0) FROM ...)` en una tabla intermedia, o usa una columna `IDENTITY`.

4.2 SCD tipo 2 en dos operaciones

Recuerda la lógica de la Clase : **UPDATE que expira la versión vigente + INSERT de la nueva versión.**

```

-- =====
-- 4.2 SP: cargar Dim_Customer (SCD tipo 2 en City, tipo 1 en Email)
-- =====
CREATE PROCEDURE gold.sp_Load_Dim_Customer AS
BEGIN
    DECLARE @Hoy date = CAST(SYSDATETIME() AS date);

    -- ---- Paso 1: SCD tipo 1 en atributos sin seguimiento histórico ----
    UPDATE tgt
    SET   tgt.Email      = src.Email,
          tgt.Segment    = src.Segment,
          tgt.RecUpdatedDate = SYSDATETIME()
    FROM  gold.Dim_Customer AS tgt
    JOIN  LH_Silver.dbo.dim_cliente_src AS src
    ON    tgt.CustomerCode = src.CustomerCode
    WHERE tgt.RecIsCurrent = 1
          AND (tgt.Email <> src.Email OR tgt.Segment <> src.Segment);

    -- ---- Paso 2: expirar las versiones vigentes cuya City ha cambiado ----
    UPDATE tgt
    SET   tgt.RecEndDate    = @Hoy,
          tgt.RecIsCurrent  = 0,

```

```

        tgt.RecUpdatedDate = SYSDATETIME()
    FROM gold.Dim_Customer AS tgt
    JOIN LH_Silver.dbo.dim_cliente_src AS src
        ON tgt.CustomerCode = src.CustomerCode
    WHERE tgt.RecIsCurrent = 1
        AND tgt.City <> src.City;

-- ---- Paso 3: insertar la nueva versión vigente ----
INSERT INTO gold.Dim_Customer
SELECT
    (SELECT MAX(Customer_SK) FROM gold.Dim_Customer)
    + ROW_NUMBER() OVER (ORDER BY src.CustomerCode),
    src.CustomerCode, src.FullName, src.City, src.Country,
    src.Segment, src.Email,
    @Hoy, CAST('9999-12-31' AS date), 1, SYSDATETIME()
FROM LH_Silver.dbo.dim_cliente_src AS src
WHERE NOT EXISTS (
    SELECT 1 FROM gold.Dim_Customer AS d
    WHERE d.CustomerCode = src.CustomerCode AND d.RecIsCurrent = 1
);
END;
GO

```

🎯 El `WHERE NOT EXISTS` cubre **los dos casos a la vez**: clientes nuevos (nunca existieron) y clientes cuya versión acaba de expirarse en el paso 2. Es el patrón estándar de SCD tipo 2.

4.3 Carga incremental del fact

```

-- =====
-- 4.3 SP: cargar Fact_Sales (incremental por línea de pedido)
-- =====
CREATE PROCEDURE gold.sp_Load_Fact_Sales AS
BEGIN
    INSERT INTO gold.Fact_Sales
    SELECT
        ISNULL(dOrd.DateKey, -1), ISNULL(dShp.DateKey, -1),
        ISNULL(p.Product_SK, -1), ISNULL(c.Customer_SK, -1), ISNULL(t.Store
_SK, -1),
        v.OrderNumber, v.OrderLine,
        CAST(v.Quantity AS int), CAST(v.UnitPrice AS decimal(10,2)),
        CAST(v.GrossAmount AS decimal(12,2)), CAST(v.DiscountAmount AS deci
mal(10,2)),
        CAST(v.NetAmount AS decimal(12,2)), SYSDATETIME()

```

```

FROM LH_Silver.dbo.ventas AS v
LEFT JOIN gold.Dim_Date AS dOrd ON dOrd.FullDate = v.OrderDate
LEFT JOIN gold.Dim_Date AS dShp ON dShp.FullDate = v.ShipDate
LEFT JOIN gold.Dim_Product AS p ON p.ProductCode = v.ProductCode
LEFT JOIN gold.Dim_Store AS t ON t.StoreCode = v.StoreCode
LEFT JOIN gold.Dim_Customer AS c ON c.CustomerCode = v.CustomerCode
                                AND c.RecIsCurrent = 1
-- Idempotencia: no reinsertar líneas ya cargadas
WHERE NOT EXISTS (
    SELECT 1 FROM gold.Fact_Sales AS f
    WHERE f.OrderNumber = v.OrderNumber AND f.OrderLine = v.OrderLine
);
END;
GO

```

4.4 Ejercicio: provocar un cambio SCD tipo 2

Vamos a simular que **Ana García se muda de Madrid a Zaragoza**.

1. Abre `NB_02_Silver_Limpieza` y ejecuta esta celda nueva:

```

from pyspark.sql import functions as F
df = spark.read.table("dim_cliente_src")
df = df.withColumn("City", F.when(F.col("CustomerCode") == "C001", F.lit(
    "Zaragoza"))
                                .otherwise(F.col("City")))
df.write.mode("overwrite").format("delta").saveAsTable("dim_cliente_src")
print("Cliente C001 movido a Zaragoza")

```

2. En `WH_Gold`, ejecuta el procedimiento y comprueba el resultado:

```

EXEC gold.sp_Load_Dim_Customer;

SELECT Customer_SK, CustomerCode, FullName, City,
       RecStartDate, RecEndDate, RecIsCurrent
FROM gold.Dim_Customer
WHERE CustomerCode = 'C001'
ORDER BY RecStartDate;

```

Punto de control 5: deben aparecer **dos filas** para C001:

Customer_SK	City	RecStartDate	RecEndDate	RecIsCurrent
1	Madrid	2024-01-01	(hoy)	0

Customer_SK	City	RecStartDate	RecEndDate	ReclsCurrent
7	Zaragoza	(hoy)	9999-12-31	1

Messages Results							
	Customer_SK	ABC CustomerCode	ABC FullName	ABC City	RecStartDate	RecEndDate	0/1 ReclsCurrent
1	1	C001	Ana García	Madrid	2024-01-01	2026-07-31	0
2	7	C001	Ana García	Zaragoza	2026-07-31	9999-12-31	1

3. Comprueba ahora el efecto sobre los hechos:

```
-- Las ventas históricas siguen apuntando a la versión "Madrid"
SELECT c.City, c.RecIsCurrent, COUNT(*) AS Lineas, SUM(f.NetAmount) AS I
mporte
FROM gold.Fact_Sales AS f
JOIN gold.Dim_Customer AS c ON c.Customer_SK = f.Customer_SK
WHERE c.CustomerCode = 'C001'
GROUP BY c.City, c.RecIsCurrent;
```

 **La lección clave:** el histórico se ha preservado. Las ventas anteriores a la mudanza siguen agregándose bajo Madrid, y solo las nuevas irán a Zaragoza. Con SCD tipo 1 habríamos reescrito la historia: todas las ventas de Ana aparecerían bajo Zaragoza como si siempre hubiera vivido allí.

Consecuencia sutil: el grano de los hechos ya no es "el cliente", sino "la **versión** del cliente". En el informe verás a Ana García dos veces si desglosas por ciudad.

5. Orquestación con un pipeline

5.1 Construir el pipeline

1. **New item** → **pipeline** → nombre `PL_Medallion`.
2. Añade las actividades **en este orden**, conectando cada una a la siguiente con la flecha verde (**On success**):

#	Actividad	Configuración
1	Notebook	<code>NB_01_Bronze_Ingesta</code>
2	Notebook	<code>NB_02_Silver_Limpieza</code>
3	Stored procedure	Connection: <code>WH_Gold</code> · Procedure: <code>gold.sp_Load_Dim_Product</code>
4	Stored procedure	<code>gold.sp_Load_Dim_Customer</code>
5	Stored procedure	<code>gold.sp_Load_Fact_Sales</code>
6	Semantic model refresh	Workspace + <code>SM_Ventas</code> (se configura tras la parte 6)
7	Outlook o Teams	Notificación de fin de carga

3. **Home** → **Save**, luego **Run**.

Punto de control 6: las siete actividades en verde en la pestaña **Output**.

★ **El orden 3 → 4 → 5 no es negociable.** Las dimensiones se cargan antes que el fact porque `sp_Load_Fact_Sales` hace lookup de la surrogate key de la **versión vigente** de cada dimensión. Si invirtieras el orden, cada línea de pedido de un cliente nuevo acabaría apuntando a `-1` (Unknown).

5.2 Notas sobre la actividad Semantic model refresh

- Es una actividad **nativa de Fabric Data Factory**; no existe en Azure Data Factory (igual que Dataflow Gen2, Teams y Outlook).
- Requiere workspace con capacidad **Premium, Premium Per User o Power BI Embedded**, y solo funciona con semantic models creados por el propio usuario.
- Por defecto ejecuta un **full refresh**; puedes seleccionar tablas y particiones concretas para hacer refresh incremental.
- **Wait on completion** viene activado: la actividad espera a que el refresh termine antes de continuar.
- En **Advanced** puedes ajustar `Max parallelism`, `Retry Count` y el modo de commit (`Transactional` o `Partial Batch`).

i Con un semantic model en **Direct Lake**, el "refresh" es un **framing**: solo actualiza metadatos apuntando a los ficheros más recientes de OneLake. Tarda segundos, no minutos. No copia datos.

5.3 Programación y disparo por eventos

- **Home** → **Schedule** para ejecución periódica.
- Alternativa: **storage event trigger**, que dispara el pipeline cuando llega un archivo al almacenamiento. Útil cuando los datos cambian de forma impredecible.

6. Semantic model — modelado y DAX

6.1 Crear el modelo en Direct Lake

1. Abre `WH_Gold` → pulsa en el icono: **New semantic model**.
2. Nombre: `SM_Ventas`. Workspace: `WS-DP600-Lab`.
3. Selecciona las tablas: `Dim_Date`, `Dim_Product`, `Dim_Customer`, `Dim_Store`, `Fact_Sales`. **No incluyas la vista `Dim_Order` por ahora.**
4. **OK** → se abre el modelado web.

◆ Al crear el modelo desde un Warehouse o desde un SQL analytics endpoint, obtienes **Direct Lake on SQL**. Si lo crearas desde el propio lakehouse o desde el OneLake catalog, obtendrías

Direct Lake on OneLake. La diferencia está en cómo se resuelve la seguridad y el acceso a los datos.

6.2 Crear las relaciones

En la vista de modelo, arrastra las claves para crear:

Desde (fact)	Hacia (dimensión)	Cardinalidad	Dirección	Estado
Fact_Sales[OrderDateKey]	Dim_Date[DateKey]	Many-to-one	Single	Activa
Fact_Sales[ShipDateKey]	Dim_Date[DateKey]	Many-to-one	Single	Inactiva ★
Fact_Sales[Product_SK]	Dim_Product[Product_SK]	Many-to-one	Single	Activa
Fact_Sales[Customer_SK]	Dim_Customer[Customer_SK]	Many-to-one	Single	Activa
Fact_Sales[Store_SK]	Dim_Store[Store_SK]	Many-to-one	Single	Activa

★ **Aquí está la role-playing dimension.** Power BI solo permite **una relación activa** entre dos tablas. La segunda queda inactiva y se activa puntualmente con `USERELATIONSHIP` dentro de una medida.

6.3 Higiene del modelo

Estas tareas parecen cosméticas pero se preguntan en el examen:

1. **Marcar la tabla de fecha:** selecciona `Dim_Date` → **Mark as date table** → columna `FullDate`. Sin esto, las funciones de time intelligence de DAX pueden dar resultados incorrectos.
2. **Ocultar las claves:** oculta `DateKey`, `Product_SK`, `Customer_SK`, `Store_SK`, `OrderDateKey`, `ShipDateKey` y todas las columnas `Rec*`. El usuario no debe verlas.
3. **Ocultar las columnas numéricas del fact** (`Quantity`, `NetAmount`, ...) una vez creadas las medidas, para forzar el uso de las medidas. (haremos esto después de crear las medidas DAX, para no perder de vista los campos mientras las escribes)
4. **Crear la jerarquía de fecha:** en `Dim_Date`, jerarquía `Calendario` con niveles `Year → QuarterName → MonthName → Day`.
5. **Ordenar los meses:** selecciona `MonthName` → **Sort by column** → `Month`. Si no, los meses se ordenan alfabéticamente.

◆ **Recordatorio Direct Lake:** este modelo **no admite columnas calculadas**. Cualquier atributo que necesites para filtrar o agrupar tiene que existir ya en las tablas Gold. Por eso creamos `YearMonth`, `QuarterName`, `DayName` e `IsWeekend` en T-SQL y no en DAX.

6.4 Medidas DAX

Crea una tabla vacía llamada `_Medidas` para agruparlas, y añade:

Medidas base

```
Ventas Netas = SUM ( Fact_Sales[NetAmount] )
```

```
Ventas Brutas = SUM ( Fact_Sales[GrossAmount] )
```

```
Unidades = SUM ( Fact_Sales[Quantity] )
```

```
Descuento Total = SUM ( Fact_Sales[DiscountAmount] )
```

Degenerate dimension en acción

```
Nº Pedidos = DISTINCTCOUNT ( Fact_Sales[OrderNumber] )
```

```
Nº Líneas = COUNTROWS ( Fact_Sales )
```

El patrón del ratio — la lección de la sección 2.4

```
% Descuento =  
DIVIDE (  
    [Descuento Total],  
    [Ventas Brutas]  
)
```

```
Ticket Medio =  
DIVIDE (  
    [Ventas Netas],  
    [Nº Pedidos]  
)
```

★ **Por qué esto funciona.** Almacenamos `DiscountAmount` y `GrossAmount` en el fact, **no el porcentaje**. Si hubiéramos guardado `DiscountPct` como columna, sumarlo o promediarlo daría resultados sin sentido. Al calcular el ratio con `DIVIDE` sobre las medidas agregadas, el porcentaje es correcto **en cualquier nivel de agregación**. `DIVIDE` además gestiona la división por cero sin errores.

Role-playing dimension con USERELATIONSHIP

```
Ventas por Fecha de Envío =  
CALCULATE (  
    [Ventas Netas],  
    USERELATIONSHIP ( Fact_Sales[ShipDateKey], Dim_Date[DateKey] )  
)
```

```

Días Medios de Envío =
AVERAGEX (
    Fact_Sales,
    DATEDIFF (
        RELATED ( Dim_Date[FullDate] ),
        CALCULATE ( MAX ( Dim_Date[FullDate] ),
            USERELATIONSHIP ( Fact_Sales[ShipDateKey], Dim_Date[DateKey] ) ),
        DAY
    )
)

```

Time intelligence

```

Ventas Netas AA =
CALCULATE (
    [Ventas Netas],
    SAMEPERIODLASTYEAR ( Dim_Date[FullDate] )
)

```

```

Variación YoY % =
VAR Actual = [Ventas Netas]
VAR Anterior = [Ventas Netas AA]
RETURN
    DIVIDE ( Actual - Anterior, Anterior )

```

```

Ventas Netas YTD =
TOTALYTD ( [Ventas Netas], Dim_Date[FullDate] )

```

Formato

- **Ventas Netas**, **Ventas Brutas**, **Descuento Total**, **Ticket Medio** → moneda, 2 decimales.
- **% Descuento**, **Variación YoY %** → porcentaje, 1 decimal.
- **Unidades**, **Nº Pedidos**, **Nº Líneas** → número entero con separador de miles.

En **Fact_Sales**, oculta (icono de ojo / clic derecho → Hide in report view):

- **Quantity**
- **UnitPrice**
- **GrossAmount**
- **DiscountAmount**
- **NetAmount**

6.5 Validación cruzada

Comprueba que el modelo devuelve lo mismo que el warehouse. En `WH_Gold`:

```
SELECT
    SUM(NetAmount) AS VentasNetas,
    SUM(Quantity) AS Unidades,
    COUNT(DISTINCT OrderNumber) AS NumPedidos,
    SUM(DiscountAmount) / SUM(GrossAmount) AS PctDescuento
FROM gold.Fact_Sales;
```

Punto de control 7: los cuatro valores deben coincidir con las medidas DAX en una tarjeta sin filtros.

7. Informe en Power BI

7.1 Crear el informe

Desde `SM_Ventas` → New report → **Start from scratch**. Nombre: `RPT_Ventas`.

7.2 Página 1 — Resumen ejecutivo

Visual	Campos	Qué demuestra
Tarjetas (4)	<code>Ventas Netas</code> , <code>Unidades</code> , <code>Nº Pedidos</code> , <code>Ticket Medio</code>	Medidas base
Gráfico de líneas	Eje: <code>Dim_Date[YearMonth]</code> · Valores: <code>Ventas Netas</code> , <code>Ventas Netas AA</code>	Time intelligence
Gráfico de barras	Eje: <code>Dim_Product[Category]</code> · Valores: <code>Ventas Netas</code>	Agregación por dimensión
Matriz	Filas: jerarquía <code>Calendario</code> · Columnas: <code>Dim_Store[Channel]</code> · Valores: <code>Ventas Netas</code> , <code>Variación YoY %</code>	Drill-down y jerarquías
Segmentación	<code>Dim_Date[Year]</code>	Filtrado
Segmentación	<code>Dim_Customer[Segment]</code>	Filtrado

Prueba el **drill-down** en la matriz: Year → Quarter → Month → Day. Esto solo funciona porque construimos la jerarquía en una **única tabla desnormalizada**. Si `Dim_Date` estuviera normalizada en snowflake, tendrías que crear una vista que la volviera a unir.

7.3 Página 2 — Logística (role-playing dimension)

Visual	Campos
Gráfico de columnas agrupadas	Eje: <code>Dim_Date[MonthName]</code> · Valores: <code>Ventas Netas</code> y <code>Ventas por Fecha de Envío</code>
Tarjeta	<code>Días Medios de Envío</code>
Tabla	<code>Dim_Store[StoreName]</code> , <code>Ventas Netas</code> , <code>Ventas por Fecha de Envío</code> , <code>Días Medios de Envío</code>

🔗 **Observa la diferencia entre las dos series.** Las mismas ventas, agrupadas por dos fechas distintas usando **una sola tabla física** `Dim_Date`. Eso es una role-playing dimension. Cambia el mes y verás cómo las barras se desplazan: los pedidos de finales de mes se envían el mes siguiente.

7.4 Página 3 — Efecto del SCD tipo 2

Visual	Campos
Tabla	<code>Dim_Customer[FullName]</code> , <code>Dim_Customer[City]</code> , <code>Ventas Netas</code> , <code>Nº Pedidos</code>
Segmentación	<code>Dim_Customer[CustomerCode]</code> filtrado a <code>C001</code>

🔍 **Ana García aparece dos filas: Madrid y Zaragoza.** Sus ventas históricas se mantienen asignadas a Madrid. Este es exactamente el comportamiento que buscábamos, pero también explica por qué **no se debe aplicar SCD tipo 2 a todos los atributos**: el modelo se llena de versiones duplicadas de la misma entidad y los usuarios se confunden.

Ejercicio de discusión con los alumnos: ¿cómo mostrarías las ventas totales de Ana consolidadas, independientemente de la ciudad? (*Pista: agrupa por `CustomerCode` o `FullName` , no por `City` — el código natural es estable entre versiones.*)

Volver al pipeline PL_Medallion para configurar las demas actividades

Actividad 6 — Semantic model refresh

1. En `PL_Medallion` , ve a la pestaña **Activities** → busca **"Semantic model refresh"** → añádela al lienzo.
2. Conéctala desde el icono verde ✓ (On success) de `05 - Fact Sales` .
3. Selecciona el bloque nuevo → pestaña **Settings**.
4. En **Connection**, haz clic en el desplegable → **Browse all**.
5. En el diálogo "Get data", selecciona la tarjeta **"Power BI Semantic Model"** (bajo "New sources").
6. En "Connect data source":
 - **Connection:** deja `Create new connection` .
 - **Connection name:** deja el valor por defecto (`PowerBIDatasets`).
 - **Data gateway:** `(none)` .
 - **Authentication kind:** `Organizational account` .
 - Pulsa **Sign in** y autentica con tu cuenta de Microsoft/organización (la misma que usas en Fabric).
 - **Privacy Level:** `None` (o el que prefieras).
 - Pulsa **Connect**.

7. De vuelta en el pipeline, en **Settings**, verifica que queden rellenos los cuatro campos:
- **Connection:** la conexión recién creada (`PowerBIDatasets <tu_usuario>`)
 - **Workspace:** `WS-DP600-Lab`
 - **Semantic model:** `SM_Ventas`
 - **Table(s):** déjalo vacío (`No results found`) para que sea un full refresh de todo el modelo.

Actividad 7 — Notificación con Office 365 Outlook (Legacy)

1. En `PL_Medallion`, ve a la pestaña **Activities** → busca "**Outlook**" → añade la actividad **Office 365 Outlook (Legacy)** al lienzo.
2. Conéctala arrastrando desde el icono verde ✓ (On success) de `Semantic model refresh` hasta este nuevo bloque.
3. Selecciona el bloque → pestaña **Settings**.
4. Pulsa **Sign in** para autenticar la conexión.
5. Aparece la pantalla de confirmación de Microsoft (*"Confirmation required — This connection was created from a different organization than yours"*) — es el aviso estándar del conector oficial de Outlook en Fabric, no indica ningún problema de seguridad real si el flujo procede del propio pipeline. Marca **"I have verified this request and trust the source"** → **Allow access**.
6. Con la sesión ya iniciada (verás *"Signed in as: tu_correo"*), rellena:
 - **To:** tu correo (para la práctica, tu propia dirección).
 - **Subject:** `PL_Medallion completado`
 - **Body:** un resumen del proceso, por ejemplo:

```
El pipeline PL_Medallion ha finalizado correctamente.
```

```
Resumen de la carga:
```

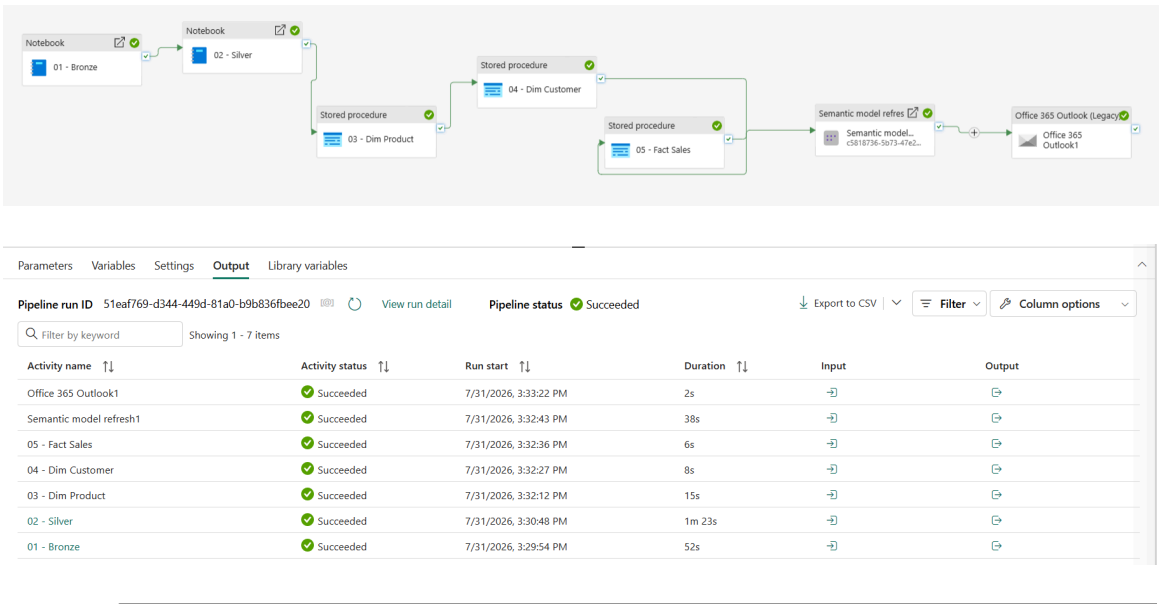
- Bronze: ingesta completada
- Silver: limpieza y conformación completada
- Gold: dimensiones y Fact_Sales cargados
- Semantic model SM_Ventas actualizado (Direct Lake framing)

7. Los campos de **Sensitivity** y **Advanced** (From, Cc, Bcc, Reply to, Importance) se dejan vacíos/por defecto — no son obligatorios.

Guardar y ejecutar

1. Ve a la cinta **Home** → botón **Save** (icono de disquete).
2. Ve a la pestaña **Run** → botón **Run**.
3. El pipeline se ejecuta de principio a fin (las 7 actividades). Puedes seguir el progreso en el panel inferior **Output**, con auto-refresh activo.

4. Al terminar, revisa que las 7 actividades queden en verde (**Succeeded**), y comprueba tu bandeja de entrada de Outlook para confirmar que llegó el correo de notificación.



From: Juan José Rodríguez Balza -3431- <juanjose.rodriguez.3431@tajamar365.com>
Sent: Friday, 31 July 2026 15:33:23
To: Juan Jose Rodriguez Balza <juanjose.rodriguez@tajamar365.com>
Subject: PL_Medallion completado

El pipeline PL_Medallion ha finalizado correctamente.

Resumen de la carga:

- Bronze: ingesta completada
- Silver: limpieza y conformación completada
- Gold: dimensiones y Fact_Sales cargados
- Semantic model SM_Ventas actualizado (Direct Lake framing)

...

[Message clipped] [View entire message](#)

9. Limpieza

Para liberar capacidad al terminar: **Workspace settings** → **General** → **Remove this workspace**.
Elimina todos los ítems de forma irreversible.

Referencias verificadas

Arquitectura y capas

- [Understand medallion lakehouse architecture for Fabric with OneLake](#)
- [OneLake shortcuts](#)
- [What is a lakehouse in Microsoft Fabric?](#)
- [Lakehouse SQL analytics endpoint use cases](#)

Modelado dimensional

- [Dimensional modeling in Fabric Data Warehouse \(overview\)](#)
- [Fact tables](#)
- [Dimension tables](#)
- [Load tables](#)
- [Slowly changing dimension type 2 \(Data Factory\)](#)

T-SQL en Fabric Data Warehouse

- [T-SQL surface area in Fabric Data Warehouse](#)
- [Limitations of Fabric Data Warehouse](#)
- [Data types in Fabric Data Warehouse](#)
- [Table constraints](#)
- [IDENTITY columns \(Preview\)](#)
- [Query the warehouse or SQL analytics endpoint \(cross-database\)](#)
- [Transactions in Fabric Data Warehouse](#)

Orquestación

- [How to copy data using Copy activity](#)
- [Activity overview \(Fabric Data Factory\)](#)
- [Use the Semantic model refresh activity](#)
- [Schedule pipeline runs](#)
- [Storage event triggers](#)

Semantic models y Direct Lake

- [Power BI semantic models in Microsoft Fabric](#)
- [Create a Power BI semantic model](#)

- [Direct Lake overview](#)
- [Develop Direct Lake semantic models](#)
- [Direct Lake in web modeling](#)
- [Create reports on semantic models in Microsoft Fabric](#)